

**SIMATS
ENGINEERING
ASSESSMENT TOOL 3**

**COURSE NAME: OBJECT ORIENTED ANALYSIS AND
DESIGN COURSE CODE: CSA1104**

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 30%

**QUESTIONS:
6**

**Total
Marks:30**

Annexure A

**Reverse Engineering
Task**

Code of Conduct:

I, 192420006 B.Harshitha (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.Harshitha

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes, service classes, and data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO, Repository, or Data Access patterns**, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/ product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/ product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction ; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

1. Legacy E-Learning Management System

An existing E-Learning Management System has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using object-oriented principles and layered architecture, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

1. Aim

To reverse engineer the existing E-Learning Management System, identify its classes, responsibilities, attributes, methods, and relationships, and redesign it using object-oriented principles and layered architecture.

2. Understanding of the Existing System

The legacy E-Learning Management System performs several major activities such as:

- Student registration
- Course management
- Learning content delivery
- Assignment submission and evaluation
- Result generation

In the existing system, user-interface code and business logic are tightly coupled. Therefore, even a small modification in one functionality may require changes in several parts of the system.

3. Problems Identified in the Legacy System

The major design problems are:

1. High coupling: UI modules directly depend on business logic.
2. Low cohesion: Individual modules perform several unrelated activities.
3. Poor maintainability: Changes to business rules may require modification of UI code.
4. Limited scalability: Adding new learning features becomes difficult.
5. Poor testability: Business functions cannot easily be tested independently.
6. Code duplication: Similar operations may occur in different modules.

4. Reverse Engineered Classes

Class	Important Attributes	Major Methods	Responsibility
Student	studentId, name, email	register(), enrollCourse(), submitAssignment()	Manages student information
Course	courseId, courseName, description	addContent(), enrollStudent()	Manages course details
Content	contentId, title, type	uploadContent(), viewContent()	Handles learning materials
Assignment	assignmentId, title, deadline	createAssignment(), submitAssignment()	Handles assignments
Evaluation	evaluationId, marks, feedback	evaluate(), provideFeedback()	Evaluates submissions
Result	resultId, grade, marks	calculateResult(), generateResult()	Generates student results

5. Redesigned Layered Architecture

The system can be divided into four layers:			
Presentation Layer:			
Contains StudentUI, CourseUI and AssignmentUI.			
Business/Service Layer:			
Contains StudentService, CourseService, AssignmentService and ResultService.			

It implements the actual business rules.

Domain Layer:

Contains Student, Course, Content, Assignment, Evaluation and Result classes.

Data Access Layer:

Contains StudentDAO, CourseDAO, AssignmentDAO and ResultDAO. These classes communicate with the database.

Architecture Flow

User → Presentation Layer → Service Layer → Domain Layer → Data Access Layer → Database

6. Design Principles Applied

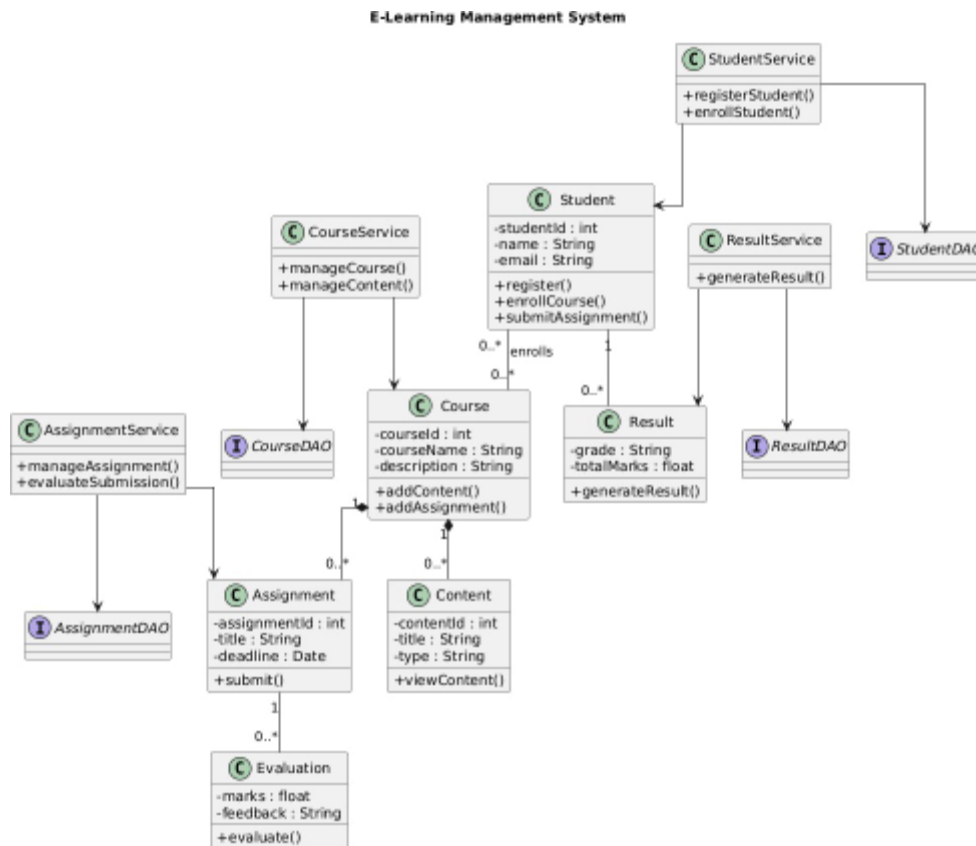
The redesign follows:

- Single Responsibility Principle: Each class performs one major responsibility.
- Open/Closed Principle: New functionality can be added without changing existing classes significantly.
- Encapsulation: Internal data is protected within classes.
- Separation of Concerns: UI, business logic and database operations are separated.
- Dependency abstraction: Business classes do not directly depend on database implementation.

7. Advantages of Redesigned System

- Improved Cohesion: Each class contains closely related operations.
- Reduced Coupling: The UI communicates with service classes rather than directly interacting with database logic.
- Maintainability: Changes in one layer have minimum effect on other layers.
- Scalability: New modules such as online quizzes, discussion forums and certification can easily be incorporated.
- Testability: Individual service and domain classes can be tested separately.
- Reusability: Business services can be reused by web, mobile or desktop interfaces.

8. UML Class Diagram Code



9. Result

Thus, the legacy E-Learning Management System is successfully reverse engineered and reconstructed using layered architecture and object-oriented principles. The redesigned system achieves better cohesion, loose coupling, maintainability, testability and scalability.

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable behavioral design patterns, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

1. Aim

To analyze the object-oriented design limitations of the legacy Customer Support System and refactor it using suitable behavioral design patterns.

2. Existing System Analysis

The customer support application receives customer requests through different channels such as:

- Email
- Chatbot
- Telephone
- Social media

The legacy system uses multiple conditional statements to select the required channel. For example:

```
if channel == email process email
```

```
else if channel == chatbot process chatbot
```

```
else if channel == telephone process telephone
```

```
...
```

Whenever a new support channel is introduced, these conditional statements must be modified across multiple modules.

3. Object-Oriented Design Limitations

The major limitations are:

- Excessive conditional logic
- Tight coupling between request handling and communication channels
- Violation of Open/Closed Principle
- Poor extensibility
- Difficult maintenance
- Repeated processing logic
- Difficult unit testing

4. Reverse Engineered Components

The common behavior can be extracted into a SupportStrategy

interface. Different channels can then provide their own

Component	Responsibility
SupportRequest	Stores customer support request
SupportContext	Selects and executes a support strategy
SupportStrategy	Defines common support operation
EmailSupport	Handles email requests
ChatbotSupport	Handles chatbot requests
TelephoneSupport	Handles telephone requests
SocialMediaSupport	Handles social-media requests

5. Design Pattern Applied: Strategy Pattern

The Strategy Pattern is suitable because different algorithms or approaches for handling customer support can be encapsulated inside separate classes.

A common interface is created:
SupportStrategy It defines:

handleRequest()

Individual strategies implement this method.

The SupportContext receives the required strategy and executes it.

6. Working of Redesigned System

Customer Request → SupportContext → SupportStrategy → Selected Support Channel

For example:

- Email request → EmailSupport
- Chatbot request → ChatbotSupport
- Telephone request → TelephoneSupport
- Social media request → SocialMediaSupport

If WhatsApp support is introduced later, a new WhatsAppSupport class can simply implement SupportStrategy.

Existing support classes do not need modification.

7. Advantages of Redesigned System

Flexibility: The support method can be selected dynamically.

Extensibility: New support channels can be introduced by implementing the common interface.

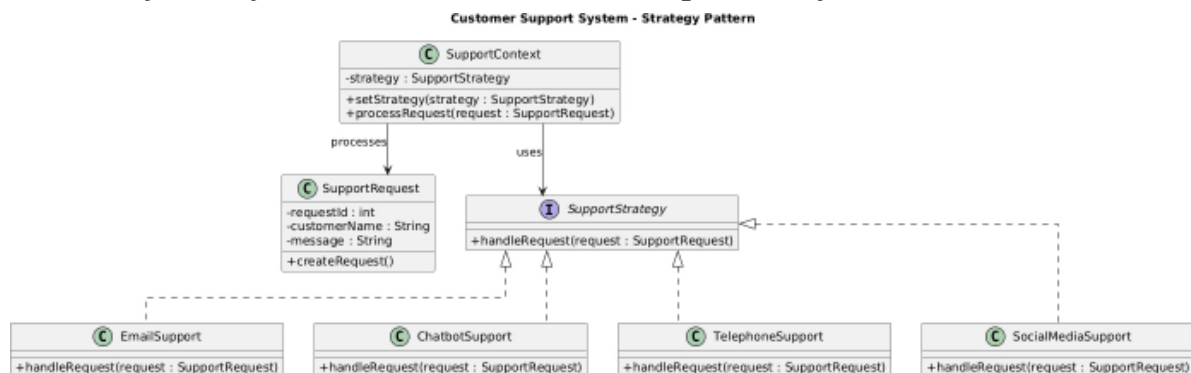
Loose Coupling: SupportContext depends on an abstraction instead of individual channels.

Open/Closed Principle: Existing classes remain unchanged when new channels are added. **Maintainability:** Each support channel contains its own processing logic.

8. UML Class Diagram

Code

Testability: Every channel can be tested independently.



9. Result

Thus, the Customer Support System is refactored using the Strategy behavioral design pattern. The redesigned architecture removes unnecessary conditional statements and provides flexibility, extensibility, loose coupling and better maintainability.

3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate domain classes, service classes, and data access components. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

1. Aim

To reverse engineer the legacy Banking Transaction System and redesign it using appropriate domain classes, service classes, data-access components, layered architecture and design patterns.

2. Existing System Analysis

The legacy banking system performs:

- Account management
- Transaction validation
- Fund transfer
- Loan processing
- Transaction reporting
- Database operations

These operations are placed within a few tightly coupled classes. Database code is also embedded inside business methods.

3. Problems Identified

The major problems are:

- Classes have too many responsibilities.
- Business and persistence logic are mixed.
- Database modifications affect business code.
- Unit testing becomes difficult.
- High coupling exists between modules.
- Reusability is limited.
- Maintenance becomes difficult as the application grows.

4. Domain Classes

Identified

Domain Class	Responsibility
Customer	Stores customer information
Account	Maintains account information
Transaction	Represents banking transactions
Loan	Represents customer loan details
Transfer	Represents fund transfer
TransactionReport	Represents transaction reports

5. Service Classes

Service	Responsibility
AccountService	Account creation and management
TransactionService	Transaction validation and execution
TransferService	Transfers funds between accounts
LoanService	Processes loans
ReportService	Generates transaction reports

6.

- AccountDAO
- TransactionDAO
- CustomerDAO
- LoanDAO

These classes provide operations such as:
save() update() delete() findById()

The service classes therefore do not contain SQL or direct database operations.

7. Redesigned Architecture

- Presentation Layer

Contains banking screens, web interfaces or mobile interfaces.

- Service Layer

Contains business operations including transactions, transfers and loan processing.

- Domain Layer

Contains Account, Customer, Transaction and Loan objects.

- Data Access Layer

Contains DAO classes responsible for database communication.

- Database Layer

Stores persistent banking records.

Flow

Banking UI → Service Layer → Domain Objects → DAO Layer → Database

8. Design Patterns Used

DAO Pattern: Separates database access from business operations. Service

Layer Pattern: Centralizes application business operations.

Repository Pattern: Can provide a collection-like interface for domain entities.

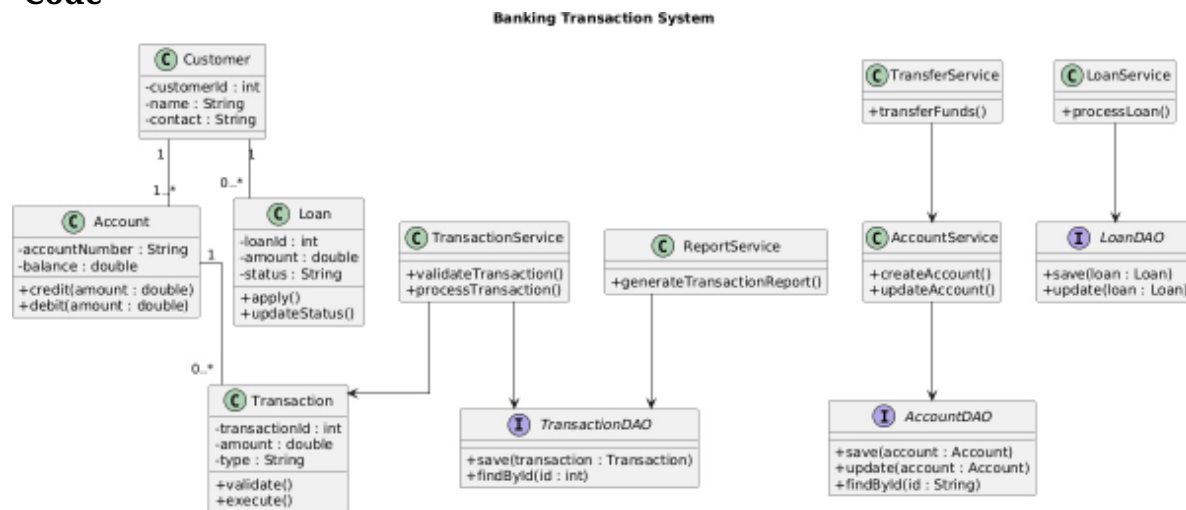
9. Improvements

Separation of Concerns: Business, UI and database responsibilities exist independently. Testability: Mock DAO implementations can be used while testing services.

Maintainability: Changes to the database require changes mainly in the DAO layer. Reusability: Banking services can be reused by web, mobile and ATM applications.

Loose Coupling: Service classes communicate through DAO interfaces instead of concrete database implementations.

10. UML Class Diagram Code



11. Result

Thus, the Banking Transaction System is reverse engineered into separate domain, service and data-access components. The redesigned architecture provides strong separation of concerns, easier testing, reduced coupling and improved system maintainability.

4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to coupling, duplication, and persistence management. Reverse engineer the system by introducing suitable DAO, Repository, or Data Access patterns, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

1. Aim

To analyze the existing Hotel Reservation System, identify coupling, duplication and persistence problems, and redesign the system using DAO and Repository/Data Access patterns for proper OODBMS integration.

2. Existing System Analysis

The legacy hotel reservation application performs OODBMS operations directly from reservation screens.

Database queries for:

- Rooms
- Guests
- Bookings
- Payments

are repeatedly written across different modules.

This creates strong dependency between the user interface and database implementation.

3. Problems Identified

- High Coupling

Reservation screens directly depend on the OODBMS.

- Code Duplication

The same room, guest, booking and payment queries appear in multiple modules.

- Poor Persistence Management

Database operations are scattered throughout UI code.

- Difficult Maintenance

Changes in the OODBMS structure require changes to several screens.

- Poor Testability

UI functions cannot easily be tested without connecting to the actual database.

4. Domain Objects

The main domain objects are:

Class	Responsibility
Guest	Stores guest information
Room	Stores room information and availability
Booking	Stores reservation details
Payment	Handles payment information

Since an OODBMS stores objects directly, these objects can be persisted without transforming them into conventional relational rows.

5. Data Access Components

Separate repository/DAO interfaces are introduced:

- GuestRepository
 - RoomRepository
 - BookingRepository
 - PaymentRepository
- Example operations include:

save() findById()

findAvailableRooms() update()

delete()

The UI no longer executes database queries directly.

6. Redesigned Architecture

Reservation UI → ReservationService → Repository/DAO →

OODBMS For example:

When a guest searches for an available room:

1. ReservationUI sends the request to ReservationService.
2. ReservationService calls RoomRepository.

3. RoomRepository communicates with the OODBMS.
4. Available Room objects are returned.
5. ReservationUI displays the results.

7. Patterns Applied

- DAO Pattern

Provides a dedicated object responsible for persistence operations.

- Repository Pattern

Provides an abstraction over the collection of domain objects stored in the OODBMS.

- Service Layer Pattern

Coordinates reservation operations between the UI and repositories.

8. Benefits of the Redesigned System

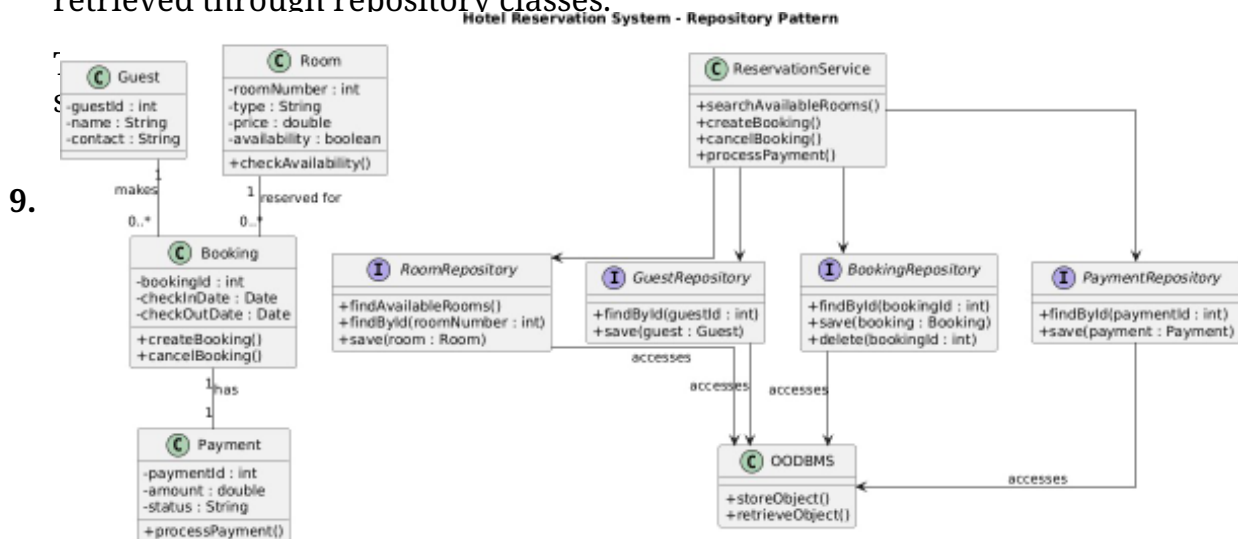
Reduced Coupling: Screens no longer depend directly on OODBMS

implementation. Elimination of Duplication: Common queries are centralized inside repositories.

Modularity: UI, business operations and data access exist as separate modules.

Reusability: Repository methods can be reused throughout the application.

Better OODBMS Integration: Domain objects can be directly stored and retrieved through repository classes.



10. Result

Thus, the Hotel Reservation System is successfully reconstructed using Service, DAO and Repository patterns. The redesigned structure reduces coupling and duplication while improving modularity, reusability, persistence management and OODBMS integration.

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable creational design patterns, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

1. Aim

To reverse engineer the Transportation Management System, identify common abstractions and object relationships, and redesign the object-creation mechanism using a suitable creational design pattern.

2. Existing System Analysis

The transportation application supports multiple transport services:

- Bus
- Train
- Taxi
- Rental vehicle

In the legacy implementation, different modules individually create these objects. For example:

```
Bus bus = new Bus() Train train = new Train() Taxi taxi = new Taxi()
```

This object creation code becomes duplicated throughout the application.

Adding another transportation option, such as Metro, may require modification in multiple modules.

3. Problems Identified

- Duplicated object creation
- Tight coupling with concrete classes
- Violation of Open/Closed Principle
- Difficult addition of new transport types
- Reduced scalability
- Difficult maintenance

4. Common Abstraction

All transportation services share common functionality. Therefore, a

Transport interface can be introduced.

It can define:

- book()
- cancel()
- calculateFare()

Classes implementing Transport include:

- Bus
- Train
- Taxi
- RentalVehicle

5. Design Pattern Applied: Factory Method

A Factory Method Pattern centralizes the creation of transportation objects.

A TransportFactory creates the appropriate object depending on the

transport type requested. Instead of directly creating:

`new Bus()`

the client uses: `TransportFactory.createTransport("BUS")`

The client therefore depends on the Transport abstraction rather than concrete classes.

6. Working

User → TransportService → TransportFactory → Transport Object

Examples:

BUS → Bus object TRAIN → Train object TAXI → Taxi object

RENTAL → RentalVehicle object

A new transport type can be added by implementing the Transport interface and extending the factory creation logic.

7. Object Relationships

- Transport is the common abstraction.
- Bus implements Transport.
- Train implements Transport.
- Taxi implements Transport.
- RentalVehicle implements Transport.
- TransportFactory creates Transport objects.
- TransportService uses Transport objects.

8. Benefits

Extensibility: New transport types can easily be introduced.

Code Reuse: Common transport behaviour is defined through one abstraction.

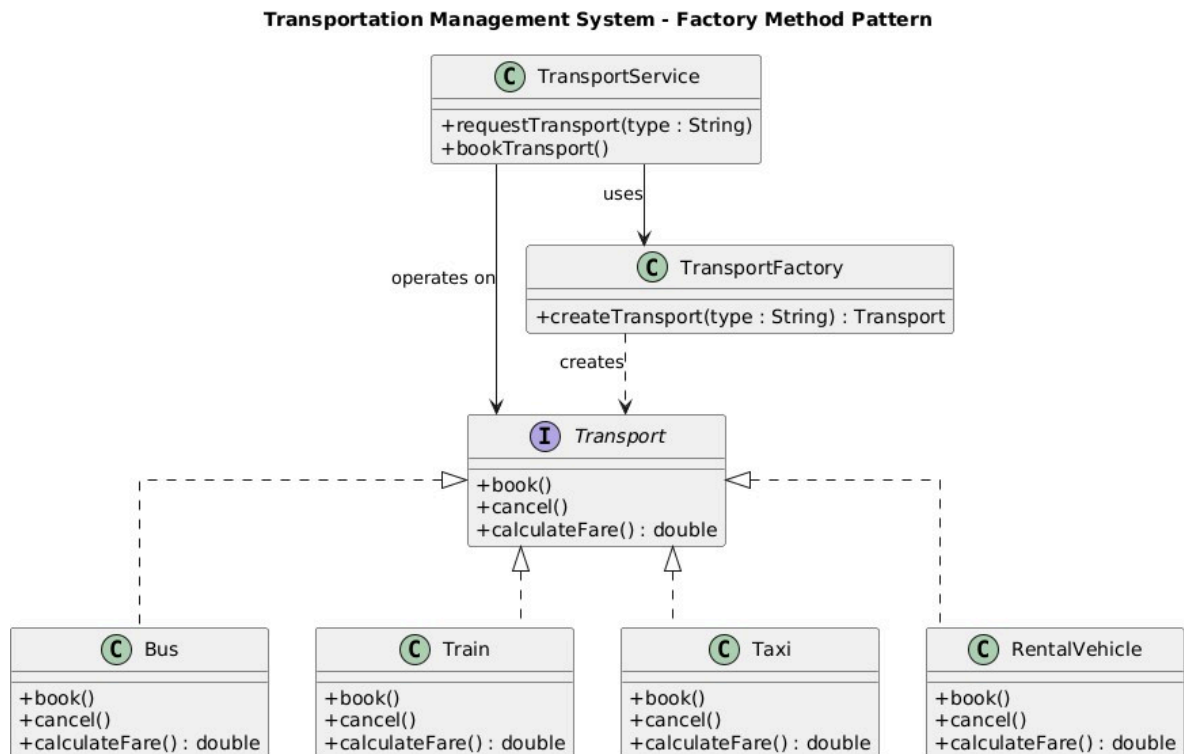
Reduced Duplication: Object creation is centralized.

Scalability: More transportation services can be supported.

Loose Coupling: Client classes depend on Transport rather than Bus, Train or

Taxi. Maintainability: Changes to creation logic are mainly managed through the factory.

9. UML Class Diagram Code



10. Result

Thus, the Transportation Management System is successfully reverse engineered using a common Transport abstraction and Factory Method pattern. The redesigned system reduces duplicated creation logic and provides better extensibility, scalability, reusability and maintainability.

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of SOLID principles, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

1. Aim

To reverse engineer the Healthcare Monitoring System, identify violations of SOLID principles, particularly SRP, OCP and DIP, and redesign the application using suitable abstractions, interfaces and design patterns.

2. Existing System Analysis

The healthcare monitoring system performs several functions:

- Collecting patient information
- Reading sensor data
- Monitoring patient conditions
- Generating alerts
- Storing medical records
- Producing healthcare reports

In the legacy system, a few classes perform several unrelated tasks and directly depend on concrete monitoring devices and storage implementations.

3. SOLID Principle Violations Single Responsibility Principle: SRP

A class should have only one major reason to change. The legacy monitoring class may:

- Read sensors
- Analyze readings
- Generate alerts
- Save records
- Produce reports

Therefore, it has too many responsibilities.

Open/Closed Principle: OCP

The system directly depends on particular sensor classes.

If a new device is added, existing monitoring code must be modified.

Therefore, the system is not properly open for extension.

Dependency Inversion Principle: DIP

High-level monitoring logic directly depends on concrete device and

database classes. For example:

MonitoringService → HeartRateSensor MonitoringService →

MedicalDatabase

Instead, both should depend on abstractions.

4. Reverse Engineered Components	
Component	Responsibility
Patient	Stores patient details
Sensor	Interface for monitoring devices
HeartRateSensor	Reads heart rate
TemperatureSensor	Reads temperature
OxygenSensor	Reads oxygen level
MonitoringService	Processes sensor information
AlertService	Generates alerts
MedicalRecordService	Manages medical records
5. Redesigned Architecture	
ReportService	Generates patient reports
Patient Domain RecordRepository	Abstracts storage operations

Contains Patient and MedicalRecord.

- Device Abstraction

A common Sensor interface is introduced.

Different devices
implement:
readData() Examples:

- HeartRateSensor
 - TemperatureSens
or
 - OxygenSensor
-
- Monitoring Service

MonitoringService receives a Sensor abstraction and processes its data.

- Alert Service

Responsible only for detecting abnormal values and producing alerts.

- Repository

RecordRepository abstracts persistent medical record storage.

- Report Service

Produces reports using stored patient information.

6. **Patterns and Principles Applied**

- Strategy Pattern

Different monitoring devices can behave as interchangeable strategies because they implement the common Sensor interface.

- Repository Pattern

The medical record storage mechanism is abstracted through RecordRepository.

- Dependency Injection

Sensor and repository dependencies are provided to service classes instead of being directly created inside them.

7. Application of SOLID Principles

SRP: Separate classes are responsible for monitoring, alerting, storage and reporting.

OCP: New sensors can be introduced by implementing Sensor without significantly modifying MonitoringService.

DIP: MonitoringService depends on the Sensor interface rather than concrete sensor classes.

8. Advantages

High Cohesion: Each component performs a clearly defined responsibility.

Loose Coupling: Services depend on abstractions.

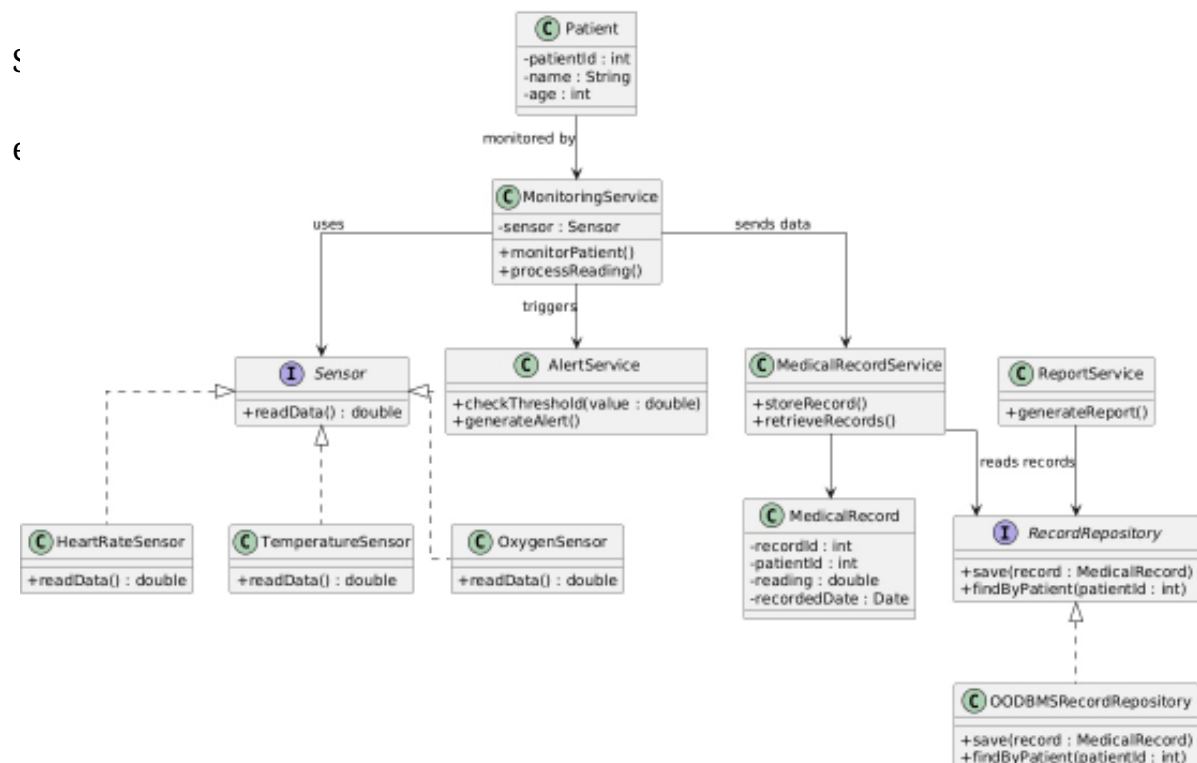
Flexibility: Different sensors and storage mechanisms can easily be

substituted. Testability: Mock sensors and repositories can be supplied

during unit testing.

Maintainability: Changes to one component have minimum impact on others

Healthcare Monitoring System - SOLID Redesign



10. Result

Thus, the legacy Healthcare Monitoring System is successfully reverse engineered by identifying violations of SRP, OCP and DIP. The reconstructed system uses abstractions, dependency injection, Strategy and Repository patterns to provide high cohesion, loose coupling, flexibility, testability and maintainability.